Recurrences
&
Master Method



The divide- and- conquer design paradigm



- Divide and conquer is just one of several powerful techniques for algorithm design.
- Divide- and- conquer algorithms can be analyzed using recurrences and the master method (so practice this math).
- Can lead to more efficient algorithms

The divide- and- conquer design paradigm



1. ***Divide*** the problem (instance) into subproblems.
2. ***Conquer*** the subproblems by solving them recursively.
3. ***Combine*** subproblem solutions.

The divide- and- conquer design paradigm

Example: merge sort

1. Divide: Trivial (array is halved).

2. Conquer: Recursively sort 2 subarrays.

3. Combine: Linear- time merge.

Recurrence for merge sort

Let T(n) = Time required for size n

T(n) =

$$2 * T(n/2) + O(n)$$

Sub-problem Size (n/2)

Number Of Subproblems	*	***Time required for each Subproblem***	+	***Work Dividing and Combining***

The divide- and- conquer design paradigm

Binary search

Find an element in a sorted array:

1. Divide: Check middle element.

2. Conquer: Recursively search 1 subarray.

3. Combine: Trivial.

Recurrence for binary search

$$T(n) = 1 * T(n/2) + \Theta(1)$$

Number Of Subproblems	*	*Time required for each Subproblem*	+	*Work Dividing and Combining*

Recurrences

- Running times of algorithms with **recursive calls** can be described using recurrences.
- A **recurrence** is an equation or inequality that describes a function in terms of its value on smaller inputs.
- For **divide and conquer** algorithms:

$$T(n) = \begin{cases} \text{solving_trivial_problem} & \text{if } n = 1 \\ \text{num_pieces } T(n/\text{subproblem_size_factor}) + \text{dividing} + \text{combining} & \text{if } n > 1 \end{cases}$$

- Example: Merge Sort

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

Solving Recurrences



- **Repeated (backward) substitution method**
  - Expanding the recurrence by substitution and noticing a pattern (this is not a strictly formal proof).
- **Substitution method**
  - guessing the solutions
  - verifying the solution by the mathematical induction
- **Recursion-trees**
- **Master method**
  - templates for different classes of recurrences

Repeated Substitution

- Let's find the running time of the merge sort

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T(n/2) + n & \text{if } n > 1 \end{cases}$$

$T(n) = 2T(n/2) + n$ **substitute**

$= 2(2T(n/4) + n/2) + n$ **expand**

$= 2^2T(n/4) + 2n$ **substitute**

$= 2^2(2T(n/8) + n/4) + 2n$ **expand**

$= 2^3T(n/8) + 3n$ **observe pattern**

$= 2^3T(n/2^3) + 3n$ **observe pattern**

assume $2^{k-1} < n \leq 2^k$.

So upper bound for k is

$n = 2^k$

$K = \log_2 n = \lg n$

$T(n) = 2^kT(n/2^k) + kn$

$= n\,T(n/n) + n \lg n$

$= n + n \lg n$

$T(n) = O(n \lg n)$

Repeated Substitution Method



- The procedure is straightforward:
  - **Substitute**, **Expand**, **Substitute**, **Expand**, …
  - Observe a ***pattern*** and determine the expression after the *k*-th substitution.
  - Find out what the highest value of *k* (number of iterations, e.g., $\lg n$) should be to get to the base case of the recurrence (e.g., *T(1)*).
  - Insert the value of *T(1)* and the expression of *k* into your expression.

Another Example...

$$T(n) = \begin{cases} 2 & \text{if } n = 1 \\ 2T(n/2) + 2n + 3 & \text{if } n > 1 \end{cases}$$

assume $2^{k-1} < n \leq 2^k$.

So upper bound for k is

$n = 2^k$

$K = \log_2 n = \lg n$

$T(n) = 2T(n/2) + 2n + 3$ **substitute**

$= 2(2T(n/4) + n + 3) + 2n + 3$ **expand**

$= 2^2 T(n/4) + 4n + 2x3 + 3$ **substitute**

$= 2^2(2T(n/8) + n/2 + 3) + 4n + 2x3 + 3$ **expand**

$= 2^3 T(n/2^3) + 2x3n + 3x(2^2 + 2^1 + 2^0)$ **observe pattern**

$T(n) = 2^k T(n/2^k) + 2nk + 3 \sum_{j=0}^{k-1} 2^j$

$= 2^k T(n/2^k) + 2nk + 3 (2^k - 1)$

$= nT(n/n) + 2n \lg n + 3(n - 1)$

$= 2n + 2n \lg n + 3n - 3$

$= 5n + 2n \lg n - 3$

$$T(n) = O(n \lg n)$$

Substitution Method



- The substitution method to solve recurrences entails two steps:
  - **Guess** the solution.
  - Use **induction** to prove the solution.
- Example:
  - $T(n) = 4T(n/2) + n$

…Substitution Method



T(n) = 4T(n/2) + n

1) Guess T(n) = O(n³), i.e., T(n) is of the form cn³

2) Prove T(n) ≤ cn³ by induction

T(n)	***= 4T(n/2) + n***	**recurrence**
	≤ 4c(n/2)³ + n	**induction hypothesis**
	= 0.5cn³ + n	**simplify**
	= cn³ – (0.5cn³ – n)	**rearrange**
	≤ cn³ if c>=2 and n>=1	

Thus *T(n) = O(n³)*

…Substitution Method

- Tighter bound for $T(n) = 4T(n/2) + n$:

  Try to show $T(n) = O(n^2)$

  Prove $T(n) \leq cn^2$ by induction

  $T(n) = 4T(n/2) + n$

  $\leq 4c(n/2)^2 + n$

  $= cn^2 + n$

  $\leq cn^2$

…Substitution Method



- What is the problem? Rewriting

  $T(n) = O(n^2) = cn^2$ - *(something positive)*

- As $T(n) \leq cn^2$ does not work with the inductive proof.
- Solution: **Strengthen the hypothesis** for the inductive proof:
  - T(n) ≤ (**answer you want**) - (something > 0)

…Substitution Method



- Fixed proof: strengthen the inductive hypothesis by subtracting lower-order terms:

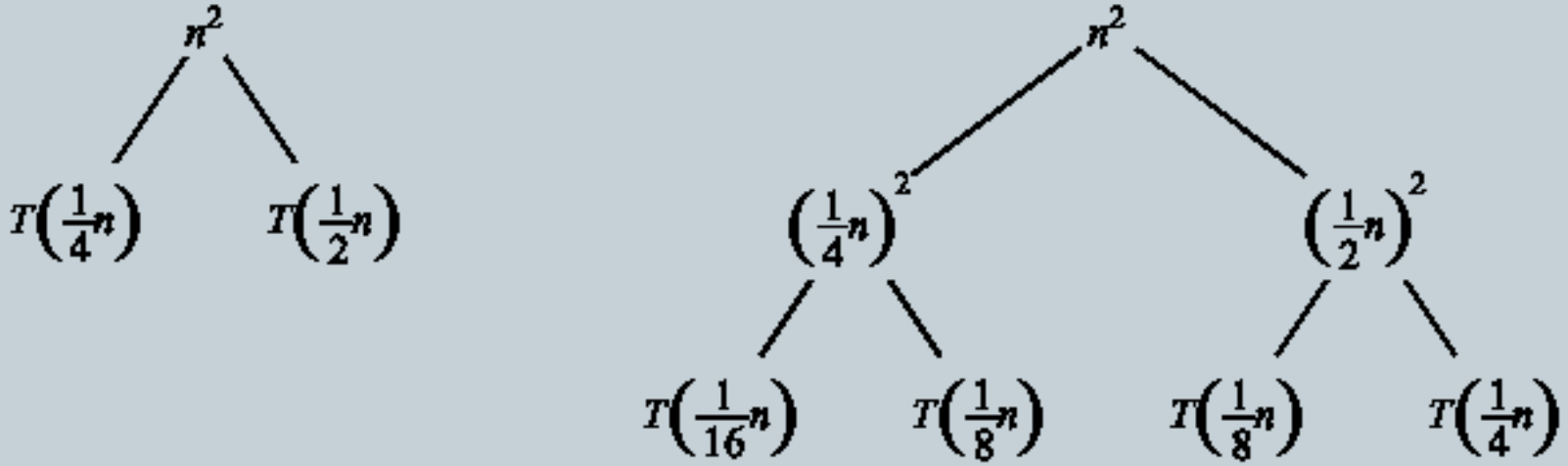
  Prove $T(n) \leq cn^2 - dn$ by induction

$$\mathbf{T(n) = 4T(n/2) + n}$$
$$\mathbf{\leq 4(c(n/2)^2 - d(n/2)) + n}$$
$$\mathbf{= cn^2 - 2dn + n}$$
$$\mathbf{= (cn^2 - dn) - (dn - n)}$$
$$\mathbf{\leq cn^2 - dn \text{ if } d \geq 1}$$

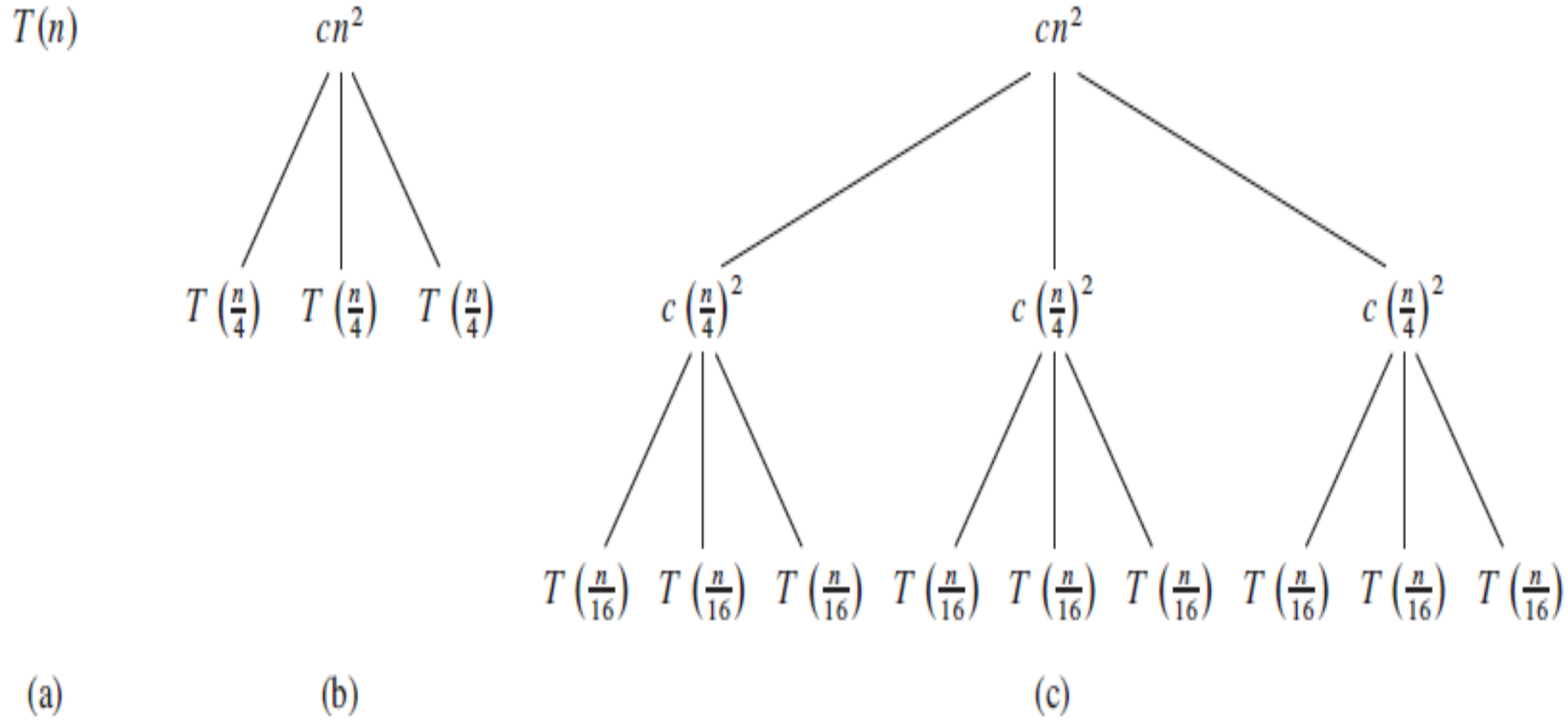
Recursion Tree

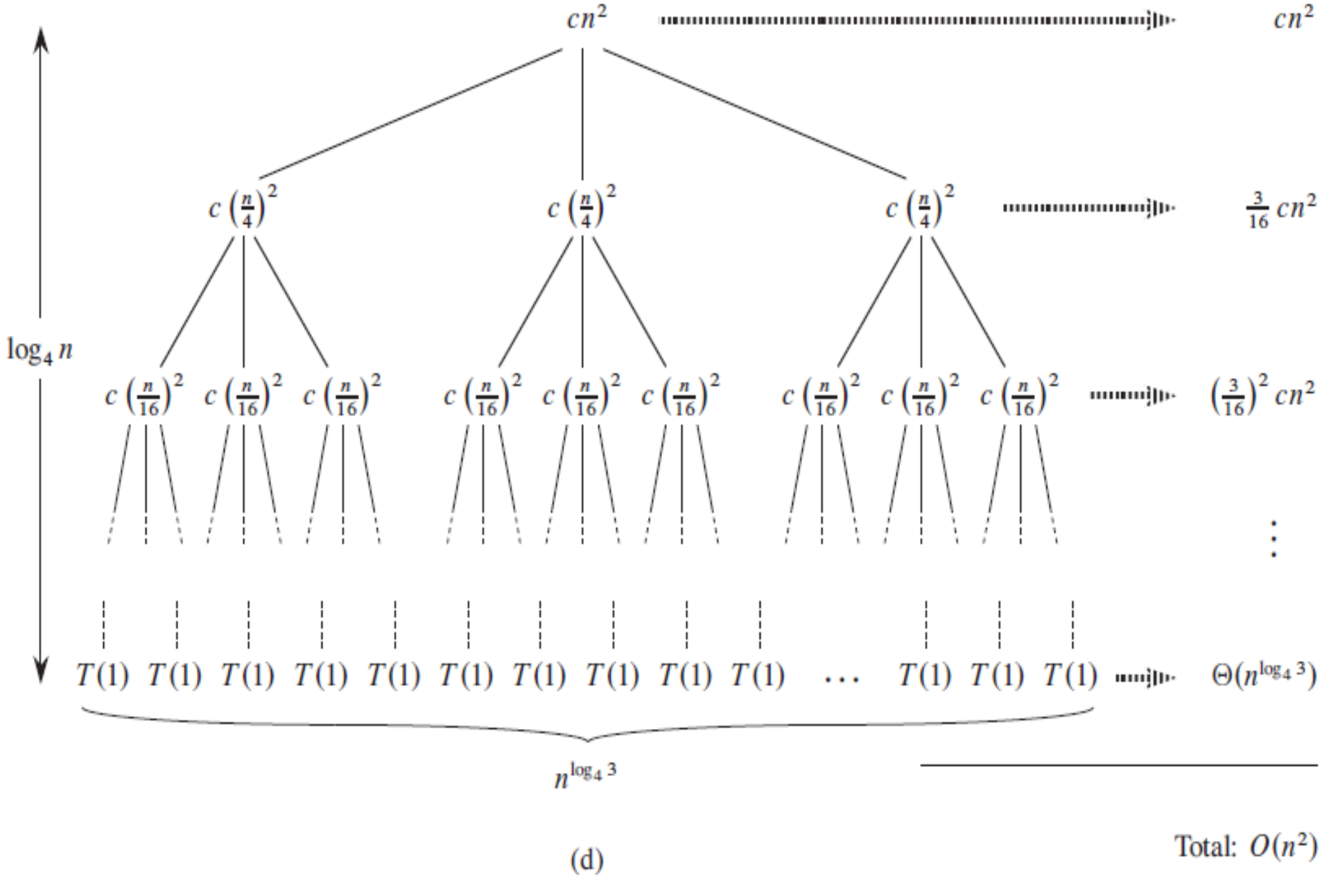
- A recursion tree is a convenient way to visualize what happens when a recurrence is iterated.
  - Good for ”guessing” asymtotic solutions to recurrences

$$T(n) = T(n/4) + T(n/2) + n^2$$



$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2) \qquad T(n) = 3T(n/4) + cn^2$$




cn^2
cn^2
$c\left(\frac{n}{4}\right)^2$
$c\left(\frac{n}{4}\right)^2$
$c\left(\frac{n}{4}\right)^2$
$\frac{3}{16}cn^2$
$\log_4 n$
$c\left(\frac{n}{16}\right)^2$ $c\left(\frac{n}{16}\right)^2$ $c\left(\frac{n}{16}\right)^2$ $c\left(\frac{n}{16}\right)^2$ $c\left(\frac{n}{16}\right)^2$ $c\left(\frac{n}{16}\right)^2$ $c\left(\frac{n}{16}\right)^2$ $c\left(\frac{n}{16}\right)^2$ $c\left(\frac{n}{16}\right)^2$
$\left(\frac{3}{16}\right)^2 cn^2$
$T(1)$ $T(1)$ $T(1)$ $T(1)$ $T(1)$ $T(1)$ $T(1)$ $T(1)$ $T(1)$ $T(1)$ $\cdots$ $T(1)$ $T(1)$ $T(1)$
$\Theta(n^{\log_4 3})$
$n^{\log_4 3}$
(d)
Total: $O(n^2)$

$$
\begin{aligned}
T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \cdots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\
&= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{(3/16)^{\log_4 n} - 1}{(3/16) - 1} cn^2 + \Theta(n^{\log_4 3}) \qquad \text{(by equation (A.5))} .
\end{aligned}
$$

Geometric series

For real $x \neq 1$, the summation

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n$$

is a ***geometric*** or ***exponential series*** and has the value

$$\sum_{k=0}^{n} x^k = \frac{x^{n+1} - 1}{x - 1} . \tag{A.5}$$

When the summation is infinite and $|x| < 1$, we have the infinite decreasing geometric series

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x} . \tag{A.6}$$

$$
\begin{aligned}
T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{1}{1-(3/16)} cn^2 + \Theta(n^{\log_4 3}) \\
&= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\
&= O(n^2) \ .
\end{aligned}
$$

Verify our guess [substitution]

$$
\begin{aligned}
T(n) &\leq 3T(\lfloor n/4 \rfloor) + cn^2 \\
&\leq 3d\,\lfloor n/4 \rfloor^2 + cn^2 \\
&\leq 3d(n/4)^2 + cn^2 \\
&= \frac{3}{16}\,dn^2 + cn^2 \\
&\leq dn^2 ,
\end{aligned}
$$

where the last step holds as long as $d \geq (16/13)c$.

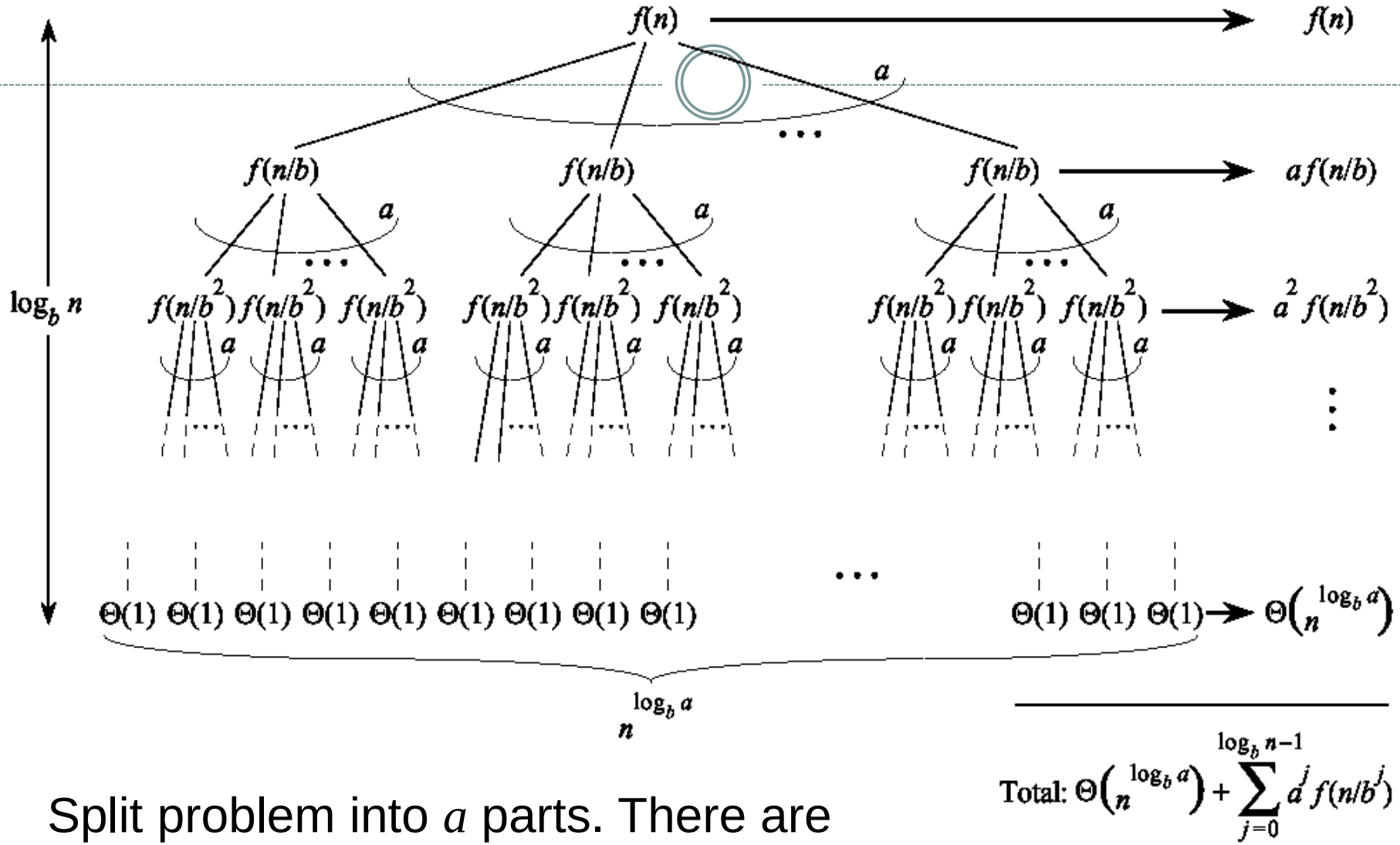
Master Method

- The idea is to solve a class of recurrences that have the form

$$T(n) = aT(n/b) + f(n)$$

- ***Assumptions:*** $a \geq 1$ and $b > 1$, and $f(n)$ is asymptotically positive.
- Abstractly speaking, $T(n)$ is the runtime for an algorithm and we know that
  - a number of subproblems of size n/b are solved recursively, each in time $T(n/b)$.
  - $f(n)$ is the cost of dividing the problem and combining the results. e.g., In merge-sort $T(n) = 2T(n/2) + \Theta(n)$

…Master Method



Split problem into a parts. There are $\log_b n$ levels. There are $a^{\log_b n} = n^{\log_b a}$ leaves.

…Master Method

- Iterating the recurrence (expanding the tree) yields

$$T(n) = f(n) + aT(n/b)$$
$$= f(n) + af(n/b) + a^2T(n/b^2)$$
$$= f(n) + af(n/b) + a^2f(n/b^2) + \ldots$$
$$a^{\log_b n-1} f(n/b^{\log_b n-1}) + a^{\log_b n} T(1)$$

$$T(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n / b^j) + \Theta(n^{\log_b a})$$

- The first term is a division/recombination cost (totaled across all levels of the tree).
- The second term is the cost of doing all subproblems of size 1 (total of all work pushed to leaves).

Master Method, Intuition



- **Three** common cases:
  1. Running time dominated by cost at leaves.
  2. Running time evenly distributed throughout the tree.
  3. Running time dominated by cost at the root.
- To solve the recurrence, we need to identify the dominant term.
- In each case compare $f(n)$ with $O(n^{\log_b a})$

Master Method, Case 1



- If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$ then
  - *f(n)* grows polynomially slower than $n^{\log_b a}$ (by factor n^{ε}).
- **The work at the leaf level dominates**

Cost of all the leaves $\Theta(n^{\log_b a})$

Master Method, Case 2

- If $f(n) = \Theta(n^{\log_b a})$ then
  - $f(n)$ and $n^{\log_b a}$ are asymptotically the same

- **The work is distributed equally throughout the tree**

  (level cost)(number of levels)

$$T(n) = \Theta(n^{\log_b a} \lg n)$$

Master Method, Case 3

- If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$ then
  - Inverse of Case 1
  - *f(n)* grows polynomially faster than $n^{\log_b a}$
  - Also need a "regularity" condition

    $\exists c < 1$ and $n_0 > 0$ such that $af(n/b) \le cf(n)\ \forall n > n_0$
- **The work at the root dominates**

  division/recombination cost

$$T(n) = \Theta(f(n))$$

Master Theorem, Summarized

Given: recurrence of the form $T(n) = aT(n/b) + f(n)$

1. $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$

   $\Rightarrow T(n) = \Theta\left(n^{\log_b a}\right)$

2. $f(n) = \Theta\left(n^{\log_b a}\right)$

   $\Rightarrow T(n) = \Theta\left(n^{\log_b a} \lg n\right)$

3. $f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$ and $af(n/b) \le cf(n)$, for some $c < 1, n > n_0$

   $\Rightarrow T(n) = \Theta\left(f(n)\right)$

Strategy



1. **Extract a, b,** and $f(n)$ from a given recurrence
2. **Determine $n^{\log_b a}$**
3. **Compare $f(n)$** and $n^{\log_b a}$ **asymptotically**
4. **Determine** appropriate **MT case and apply it**

Merge sort: $T(n) = 2T(n/2) + \Theta(n)$

1. $a=2, b=2, f(n) = \Theta(n)$
2. $n^{\log_2 2} = n$
3. $\Theta(n) = \Theta(n)$

➔ **Case 2: $T(n) = \Theta(n^{\log_b a} \log n) = \Theta(n \log n)$**

Examples of Master Method

```
BinarySearch(A, l, r, q):
    m := (l+r)/2
    if A[m]=q then return m
    else if A[m]>q then
         BinarySearch(A, l, m-1, q)
    else BinarySearch(A, m+1, r, q)
```

$T(n) = T(n/2) + 1$

1. $a=1, b=2, f(n) = 1$
2. $n^{\log_2 1} = 1$
3. $1 = \Theta(1)$

➔ Case 2: T(n) = $\Theta(\log n)$

…Examples of Master Method



$T(n) = 9T(n/3) + n$

1. $a=9,\ b=3,\ f(n) = n$
2. $n^{\log_3 9} = n^2$
3. $n = \Theta(n^{\log_3 9 - \varepsilon})$ with $\varepsilon = 1$

➔ Case 1: $T(n) = \Theta(n^2)$

…Examples of Master Method

$T(n) = 3T(n/4) + n \log n$

1. $a=3, b=4, f(n) = n \log n$

2. $n^{\log_4 3} = n^{0.792}$

3. $n \log n = \Omega(n^{\log_4 3 + \varepsilon})$ with $\varepsilon = 0.208$

➔ **Case 3:**

regularity condition: $af(n/b) <= cf(n)$

$af(n/b) = 3(n/4)\log(n/4) <= (3/4)n\log n = cf(n)$;with $c=3/4$

$T(n) = \Theta(n \log n)$

References & Readings



- CLRS
  - Chapter: 4 (4.1-4.3)
  - 4.4 for bedtime reading
  - Exercises
    - 4.1, 4.2, 4.3
  - Problems
    - 4-1, 4-4
- HSR
  - Chapter: 3 (3.1-3.6)
  - Examples: 3.1-3.5
  - Exercises: 3.1 (1, 2), 3.2 (1, 3-6),